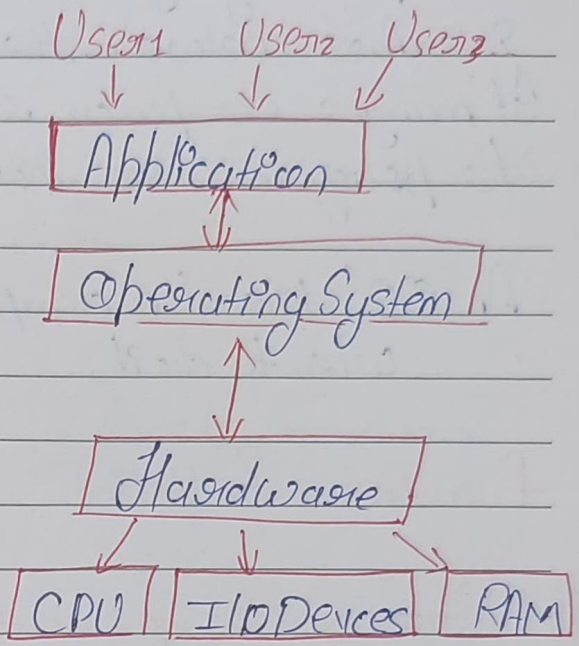
1. Concept of Operating Systems :



Eg: Windows, → use from convenience
* Throughput → (Linux)

Functions : Resource Mgmt., Process Mgmt., Storage mgmt
CPU scheduling
secondary mgmt → HD
Memory Mgmt, Security & priv
RAM
file system

What is OS & use & functions

An Operating System is system software that acts as an interface between the user & the computer hardware. It manages computer resources & provides service to program so that the computer work efficiently.

* An OS tells the computer what to do how to do it & when to do it.

functions & Uses :

i) Process Management: Controls execution of programs. Run multiple program at the same time

ii) Memory Manag. : Allocates & deallocates memory to prog. Ensures prog do not interfere with each other.

iii) File Manag. : Create, reads, write, deletes files. Organizes data into folder / directories

iv) Device Mang : Controls hardware device like Keyboard mouse, printer, etc. Uses divers to communicate with hard drive

vi) User Interface : Provides Graphic User Interface or command line Interface (CLI)
Eg: Windows desktop, Linux terminal.

vii) Security & Protection : Protects system from unauthorized access.
Uses password & permissions.

viii) Error Detection & Handling : Detects hardware & Software errors.
Takes corrective actions.

Eg Of OS : Windows, Linux, MacOS, Android, iOS

* Generation Of OS :

1) First Generation OS [1940 - 1955]

No Operating System
- Computers used vacuum tubes
- Programs were written in machine language
- No Os Concept existed
- One program executed at a time
- Input/output done using punch cards

Eg → ENIAC, UNIVAC

* Second Generation OS [1955-1965]

- Batch processing System
- Used transistors
- Jobs were collected & executed in batches
- No user interacting during execution
- Improved CPU utilization

Eg:- IBM 7094, OS Type Batch OS

* Third Generation OS [1965-1980]

- Multiprogramming & Time Sharing System
- Used Integrated Circuits (ICs)
- Multiple programs loaded into memory
- CPU switched between programs
- Allowed Multiples users simultaneously

Eg:- UNIX (early versions), IBM System/360
OS Type :- Time-sharing OS

* Fourth Generation OS [1980-2000]

*
Personal Computer Operating Systems

- Used microprocessors
- Graphic User Interface (GUI) introduced
- Supported networking & multitasking

Eg:- MS-DOS, Windows 95/98, Mac OS.

* Fifth Generation OS [2000-Present]

* Modern & Intelligent OS
- Support multicore processors
- Advance security features
- Support mobile, cloud & AI-based system
- High performance & real-time processing

Ex: Windows 10/11, Linux, Mac OS, Android, iOS

* What is Computer :

A computer is an electronic machine that takes input (data), processes it, stores it & gives output (information) according to given instruction (called prog.)

Computer → Input → Process → Output

Types of OS

An Operating System is system software that manage hardware & software resources of a computer

There are different ~~system~~ type OS based on how they work.

* Batch OS
Jobs are collected in batches & executed one by one without user interaction.

* Time-Sharing OS
CPU time is shared among many users. Each user gets small time slice, CPU switches very fast, feels like everyone is working at the same time.

* Multiprograming OS
More than one program is kept in memory & CPU switches between them. when one job wait (I/O), CPU runs another job Increases CPU utelization.

* Multitasking OS
User can run multiple tasks at the same time.
listening to music + browsing + coding

* Multiprocessing OS
Uses more then one CPU (Processor). Tasks divided among CPU Parallel execution.

* Real-Time OS:
Give quick & fixed time response Time is very critical.

* Distributed OS:
Multiple computer connected & work like one system. Resource sharing faster Processing.

* Network OS
Manages network resources & communication between computers. file sharing, Printer sharing, Security.

Operating System Services

* Program Execution service allows the OS to load a program into memory, start its execution and run it properly & terminate it after completion. The OS ensures that the program gets all required resource like CPU time, memory & input/output devices

* In/Output OS

This service allows the Operating System to manage communication btw the computer & external devices such as keyboard, mouse, pointer monitor & storage devices. It provides a uniform interface so program do not need to directly control hardware.

File System Manipulation :

The file system manipulation service allows the OS to create, delete, read, write rename & Organize files & directories on storage devices. It also controls file permissions & access right.

* Resources Allocation * Protection & Security Service
* Accounting

Communication Service

Communication service enable data exchange between processes on computer. It allows program to share information using msg passing or shared memory, both locally & over network.

Error Detection Service :

Error detection service help to operating system to detect, identify & handle hardware & software error to ensure system reliability & prevent system crashes.

Resource Management : The OS allocates CPU, memory & devices to program

Protection & Security : The OS protects data & System resources from unauthorized access

User Interface : The OS Provides an inteface for users to interact with computer

Ex : CLI & GUI

Command line Interface & Graphical user interface

Layered OS

A layered Operating system divides the OS into multiple layers, where each layer performs a specific function & interacts only with the adjacent layes

User → Application Prog → Shell → Kernel → HW

User → End users interact with the system
Uses application :- Eg: Student using Chrome, MsWord

Application layer :- Contains user program
Requests services from OS

Eg:
Browser asks OS to acces Internet
Media player asks OS for audio device

Shell / user Interface:
Interface between user - OS
Converts User commands into system calls
Eg: GUI : windows desktop

Kernel layer :- Process Management, Memory Manag. file System, Device drivers

Eg: CPU Schedulling, RAM allocation.

Hardware layer :- Physical components

Eg:- CPU, RAM, Keyboard, HD.

*What is Monolithic OS?

In a Monolithic OS all OS services run in kernel space as a single large program.

Ex:-

User → System Calls → Monolithic Kernel → Hardware
[Process + Memory + File + Device Drivers]

Services Included :- Process Manag., Memory Manag. File System, Device drivers.

* What is Microkernel OS

A microkernel OS keeps only essential service in the ~~rene~~kernel, while other services run in user space.

UserApp → Userspace Services → Microkernel → Hardware
[FileSystem, Device drivers] [IPC, Scheduling, Memory]

* Service in Microkernel : Only minimal function

Inter-Process Communication (IPC)
Scheduling
Basic Memory management.

- VMware ESXi (This means it installs directly on hardware (on Windows / Linux)
- Virtual Box (It runs on top of an existing OS (Windows / Linux / MacOS)

* What is Virtual Machine?

A Virtual Machine is software-Based computer that runs inside a physical computer.

One physical system → Multiple virtual Systems

VM Architecture.

Apple → Guest OS → VM Monitor (Hypervisor) → Host Hardware.

~~1) Type~~

Hypervisor : Controls Virtual Machines

Type 1: (Bare Metal)

Runs directly on Hardware

Ex. VM ware ESXI.

Type 2: (Hosted)
Runs on host OS
Ex. Virtual Box.

new → Ready → Running → waiting/Blocked → terminated

Processes: A process is a program in execution including the current value of the program Counter, registers & ~~Halu~~ variables.

New → ready → running → waiting → terminated

The difference between a process & a program is that the program is the group of instruction where as the process is the ~~act~~ activity.

We write our computer programs in a text file & when we execute this program it becomes a process which perform all the tasks mention in the program.

When a program is loaded into the memory & it become a process, it can be ~~di~~ divided into four section stack, Heap, text & data

i) Stack: The process stack contains the temporary data such as Method function, Parameters return address & ~~value~~ local variables

Method funtion
Parameters
return addres & local variables.

ii) Heap: This is dynamically allocated memory to process during its ~~run~~ runtime.

iii) Text: Program counters & content of processor Registers.

iv) Data: Global & Static Variables

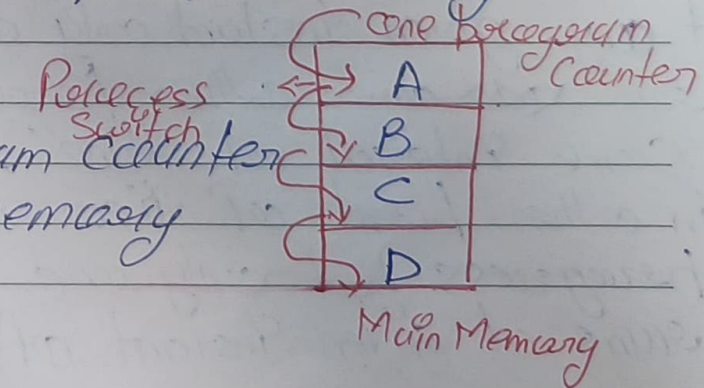
Process Model:

Conceptually, every process has its own virtual CPU but in reality CPU switches back & forth from process to process. but to understand the system it is much easier to think about a collection of processes running in (pseudo) parallel than to try to keep track of how the CPU switches from program to program. This rapid switching back and forth is called Multiprogramming.

First Model: Multiprogramming:

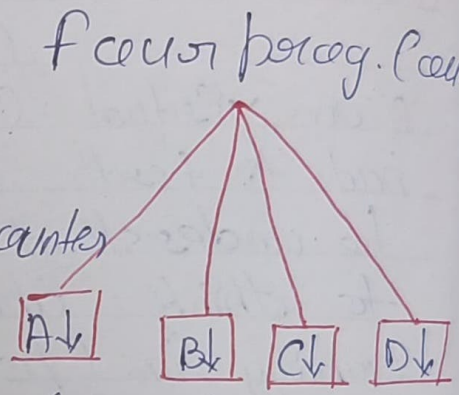
In this there is fewer programs in memory We have single shared processor by all programs.

There is only one program counter for all programs in memory



In this first - Program Counter is Initialized to execute the part of program A then with the help of process switch it transfer control to program & execute same Process to C & D (main memory)

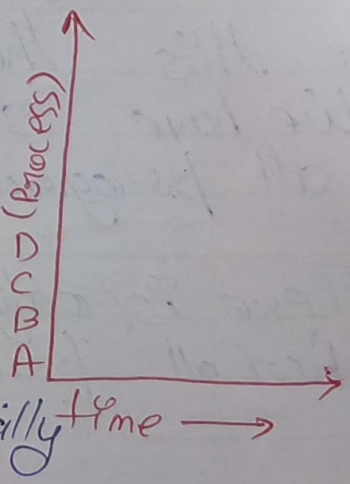
Second Model: Multi processing



There is 4 processes each with its own flow of control & each one own logic program counter

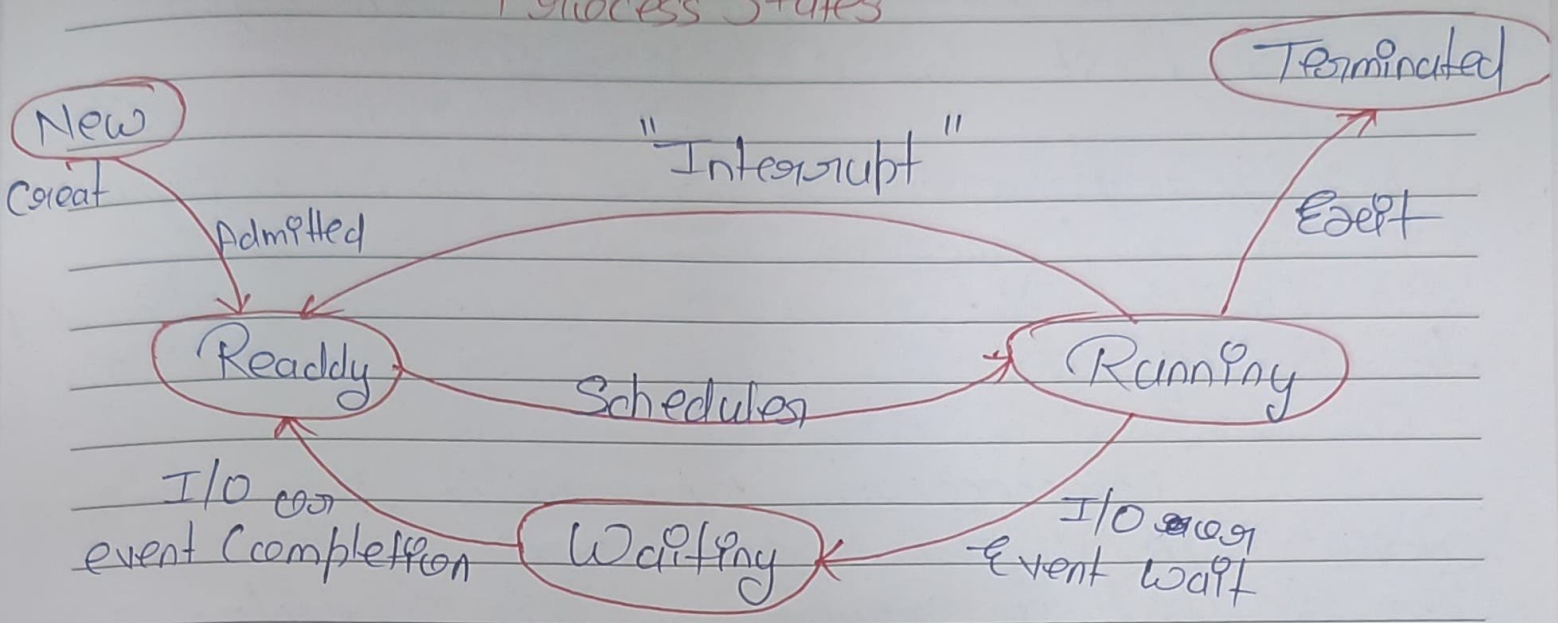
One running independently of the other ones there is only one physical counter. So when each process runs. its logical program counter is loaded into the real program counter when it is finished the physical program counter is saved in the process.

Third Model: One program at One



At any given instant only one process run and other process after some interval of time.
In other point of view all processes progress but only one process actually runs at given instant of time.

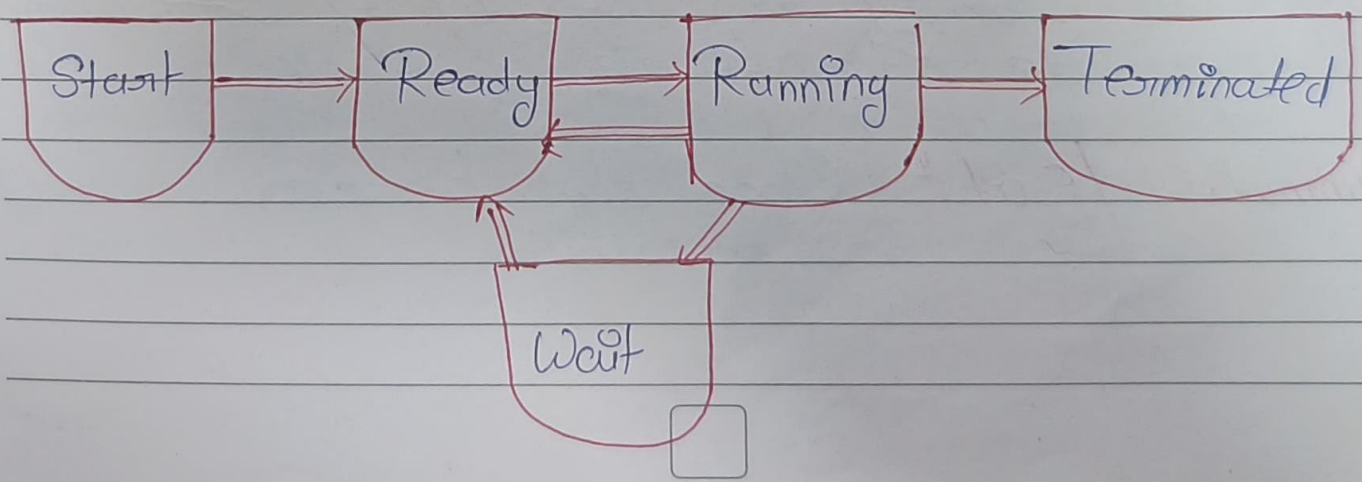
Process States



Process life Cycle

When a process executes, it passes through different state These stages may differ in different operating system.

In General a process Can have one of the following five states at a time.



Start: This is the initial state when a process is first started/ create

Ready: The process is waiting to be assigned to a processor Ready Processes waiting to have the processor. allocated to them by the operating system so that they can run.

Process may come into this states after "START" state or ~~wait~~ while running it by but interrupted by the scheduler to assign CPU to some other process.

Runing: After Ready state, the Process state is set to running & the Processor execute is it instruction.

Waiting: Process moves into the waiting state if it needs to wait for a resource such as waiting for user input or waiting for a file to become available.

Terminated / exit: Once the process finished its execution.

Threads:

Threads are small units of execution within a process that can run concurrently with other threads.

They allow for multitasking within a single program, enabling different tasks to be performed simultaneously.

i) Concurrency: Threads enable concurrent execution of tasks, improving program efficiency by utilizing available CPU Resources effectively.

ii) Resource Sharing: Threads share memory space within a process allowing for efficient communication & data sharing between task

iii) Light weight: Threads are light weight compared to processes as they share resources & memory space. This makes them faster to create & switch between.

Differen. btw Process & Thread

Process	Thread
Process is a heavy weight cost resource	Thread is light weight taking less resource than process
Process switching needs interaction with operating system.	Thread switching does not need to interact with OS
In multiprocessing environment each process executes the same code but has its own memory & file resources	All threads can share same set of open file, child processes.
If one process is blocked then no other process can execute until the first process is unblocked	While one thread is blocked & waiting second thread in the same task can run
Each process operates independently of the others in multiprocess system	One thread can read write or change another thread's data.

CPU Scheduling

CPU scheduling is a fundamental concept in operating systems that deals with the order in which processes are executed by the CPU. It's a system for managing how multiple processes share the CPU

What is CPU Scheduling

Definition: CPU scheduling is the process of the available processes in the ready queue will get the CPU for execution.

Objective: The main goal of CPU scheduling is to maximize the use of the CPU & to ensure that all processes are executed in a fair manner.

How CPU Scheduling Works?

Ready Queue: Process that are ready to execute but are not using the CPU are placed in a queue which is known as the ready queue.

Scheduler: It is a component which decides which process should be executed next based on a particular scheduling algorithm.

Context Switching: When the CPU switches from execution one process to another a context switch occurs, which involves saving the state of the old process & load the state of the new process.

Type of CPU Scheduling.

In an operating system preemptive & non-preemptive scheduling are two method used to decide how the CPU's time is divided among various processes.

Preemptive Scheduling

- In preemptive scheduling, the operating System can disturb a process that's currently using the CPU & assign the CPU to another process.
- This disturbance can happen for several reason such as when a higher-priority process arrives or a time limit (quantum) is reached.

It's like a teacher who can stop a student in the middle of a presentation if there's an urgent question from another student.

Example: of preemptive scheduling algorithms include Round Robin, Shortest Remaining Time first, & Priority (Preemptive version)

Non-Preemptive Scheduling

Non-Preemptive scheduling, once a process start using the CPU, it cannot be stopped until it finishes its execution means it can't be disturbed by any other process
The CPU remains with the process until it gives up control of the CPU willingly.

- When process complete its execution
- When process want to perform same I/O operation

This is like letting a student finish their presentation without disturbance even if other student have question
Example of non-preemptive scheduling algorithms include shortest Job first & Priority (non-preemptive version)

CPU Scheduling Performance Criteria

In a OS, Scheduling criteria are used to check the performance of CPU scheduling algo. Here's a simple explanation of each criterion along with it's formula.

- **Turnaround time** : It is the total time taken to complete a process, from submission to completion. The goal is minimiz this so that each tasks finish as quickly as possible.
  * Turnaround time = Completion time - Arrival time

- **Waiting time** : This is the total time a process spend waiting in the ready queue before it get CPU.
  * Waiting Time = Turnaround time - Burst time

- **Response time** : This is the time from the submission of a request until the first response is produced.
  * Response T. = Time of first CPU Allocation - Arrival time

- **Completion time** : This is the time when a process finishes execution & is ready to exit or switch to the waiting state.

CPU & I/O Burst Cycle

In operating system CPU burst cycle & I/O burst cycles are terms used to describe the pattern of processing & waiting times for processes.

CPU Bursts:

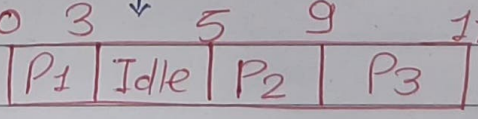
A CPU burst is when a process is using the CPU for execution.

It's like a race where the process run as fast as it can, for execution the instruction.

Idle Time: The period during which the CPU is not executing any process & remains free is called Idle Time.

P	AT	BT
P1	0	3
P2	5	4
P3	6	2

GIC :



Coppera Aarival Time Mode Non - Prem

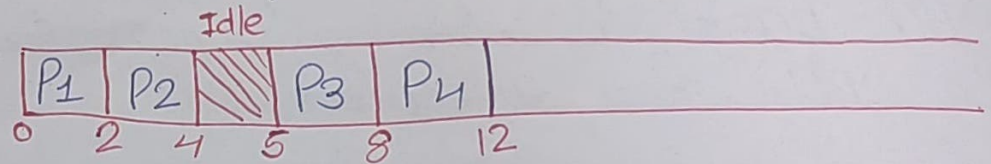
Parul University | NAAC A++

Faculty of Engineering & Technolo

Algorithm 1st first Com first Serve

Process No.	A. time	B. T	C. T	TAT	WT	RT
P1	0	2	2	2	0	0
P2	1	2	4	3	1	1
P3	5	3	8	3	0	0
P4	6	4	12	6	2	2

Gantt Chart



$TAT = CT - AT$

$WT = TAT - BT$

RT =

Respons Time =

Turn around time

avg WT = $\frac{0+1+0+2}{4}$

$= \frac{3}{4} = 0.75$

avg TAT = $\frac{2+3+3+6}{4}$

$= \frac{14}{4} = 3.5$

Ready qupe → A:T

Burst Time = How much time take from the process

On the case of non-preemption the waiting time same Reponce time.

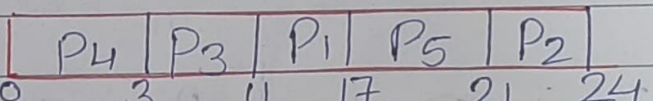
Criteria : Arrival time

Mode : Non-preemption

Calculate average waiting time

Process	AT	BT	CT	CT-AT TAT	TAT-BT WT	P_i-AT RT
P1	2	6	17	15	9	9
P2	5	3	24	19	16	16
P3	1	8	11	10	2	2
P4	0	3	3	3	0	0
P5	4	4	21	17	13	13

Grantt chart :



$$\text{Avg WT} = \frac{9+16+2+0+13}{5}, \quad \text{Avg TAT} = \frac{15+19+10+3+17}{5}$$

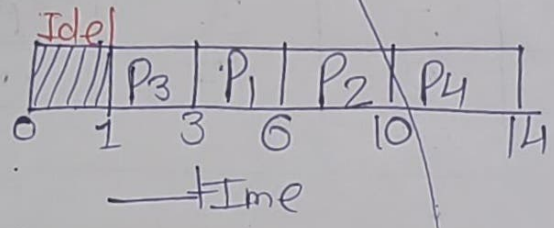
$$= \frac{40}{5} = 8 \qquad = \frac{64}{5} = 12.8$$

ii) Shortest Job First (SJF)

Criteria = Burst time Mode Non-Preemptive

Process	AT	BT	CT	TAT (CT-AT)	WT (TAT-BT)	RT
P1	1	3	6	5	2	2
P2	2	4	10	8	4	4
P3	1	2	3	2	0	0
P4	4	4	14	10	6	6

Gantt Chart



Avg BT = $\frac{3+4+2+4}{4}$

$= \frac{13}{4} = 3.25$

Avg CT = $\frac{6+10+3+14}{4}$

$= \frac{33}{4} = 8.25$

Avg TAT = $\frac{5+8+2+10}{4}$

$= \frac{25}{4} = 6.25$

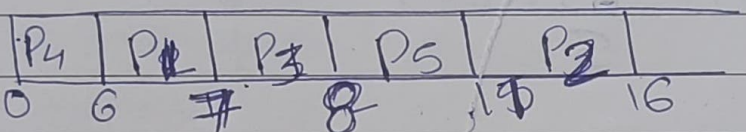
Avg WT = $\frac{2+4+0+6}{4}$

$= \frac{12}{4} = 3$

Out of all available (waiting) processes it selects the process with the smallest burst time to execute Next.

Process	AT	BT	CT	TAT	WT	RT
P_1	2	1	7	5	4	4
P2	1	5	16	15	10	10
P3	4	1	8	4	3	3
P4	0	6	6	6	0	0
P5	2	3	11	9	6	6

GC



$$\text{Avg TAT} = \frac{5+15+4+6+9}{5} = \frac{39}{5} = 7.8$$

* The switching between two processes A & B need PCB to save the state

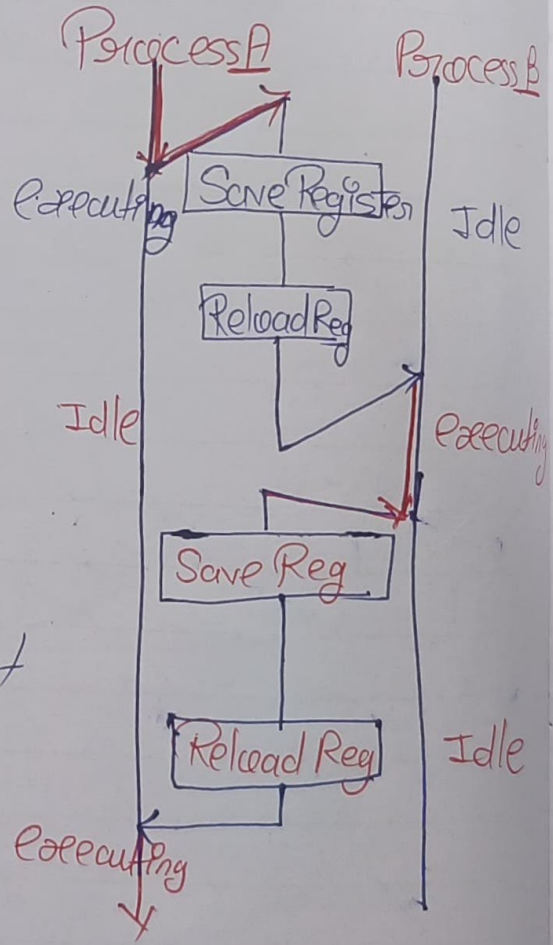
Process : Context Switch.

A context switch (also sometimes referred to as a process switch or a task switch) is the switching of the CPU from one process or to another.

A context is the contents of a CPU's registers & program counter at any point in time.

Context Switch - Steps

If two Processes A & B are in ready queue.
If CPU is executing process A & process B in wait state
If an interrupt occurs for process A, the OS suspends the execution of first process & stores the store the current info. of process A in its PCB & context to the second Process namely Process B



In doing so, program counter from the PCB of Process B is loaded & thus execution can continue with the new process

Algo 3 :- Round Robin :

Each selected process is assigned a time interval called time quantum or time slice.

Here 2 things are Possible :-

If Process is blocked or ~~time~~ terminated before the 'time slice' has elapsed, then CPU Switching is done & another Process is Scheduled to run

If process needs CPU Burst longer then 'Time Slice' then the Process is running at the end of ~~the~~ of time quantum.

Now that process will be Preempted & move to end of Ready queue & CPU will be allocated to process which in front of ready queue.

Particular time interval => Time Quantum

22
12
34
40

Parul® University | NAAC A++

Faculty of Engineering & Technology

Given Time Quantum = 2

P.ID	AT	BT	CT	CT-AT TAT	TAT-BT WT	CT-TAT RT
P1	0	8,1,3,	12	12	7	0
P2	1	2	4	3	1	1
P3	2	1	5	3	2	2
P4	3	4	14	11	7	3
P5	4	3	15	11	8	4

GC: P1

Raddy → | P1 | P2 | P3 | P1 | P4 | P5 | P1 | P4 | P5 |

0 2 4 5 7 9 11 12 14 15

GC | P1 | P2 | P3 | P1 | P4 | P5 | P1 | P4 | P5 |

0 2 4 5 7 9 11 12 14 15

$$WT = \frac{7+1+2+7+8}{5} = \frac{25}{5} = 5$$

$$TAT = \frac{12+3+3+11+11}{5} = \frac{40}{5} = 8$$

Ready queue

→ | P1 | P2 | P3 | P1 | P3 | P4 | P5 | P1 |

0

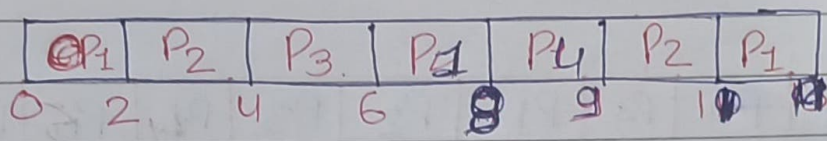
→ | P1 | P2 | P3 | P1 | P3 |

Run 0 2 4 5 7

Given TQ = 2

				CT-AT	TAT-BT	CT-AT
Pelo	AT	BT	CT	TAT	WT	RT
P_1	0	5	12	12	7	0
P_2	1	4	10	10	6	1
P_3	2	2	6	4	2	2
P_4	4	1	9	5	4	4

GC :



WT Avg :- $\frac{7+6+2+4}{4} = \frac{19}{4} = 4.7$

TAT Avg :- $\frac{12+10+4+5}{4} = \frac{21}{4} = 5.2$

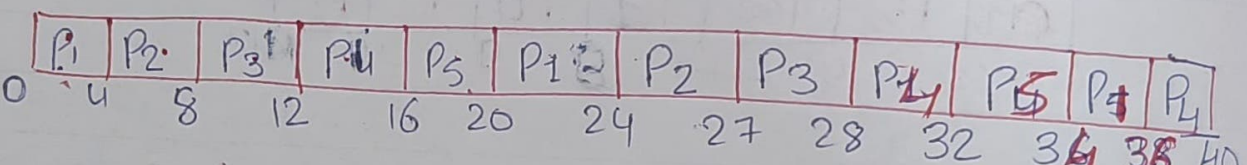
Reddy →

Running →

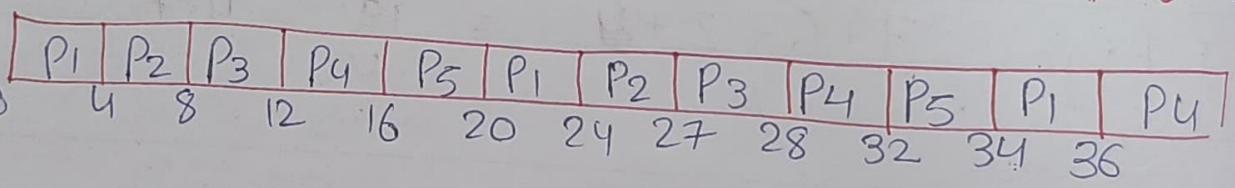
Given TQ = 4

Proc	AT	BT	CT
P1	0	12	32
P2	1	7	27
P3	2	5	28
P4	3	10	40
P5	4	6	38

GC



GC



Unit - 3

Inter Process Communication

It is a mechanism provided by the OS that allows process to communicate with each other known as IPC

Process executing concurrently in the OS may be either independent process or co-operating process.

In OS we have two or more process that are executing at the same time in the OS this

* **Independent Process :**

They can't affect or can't affected by the other process executing in the system.

* **Co-Operating process :**

They can affect or be affected by the other process executing in the system.

Ex: any process that shares with other process if process is sharing data with another process then we can say for sure that process can.

can affect the other process which is sharing the data so if two processes are sharing data than those kind of processes are known as co-operating process.

——×——×——

Why we need IPC?

There are several reasons which allows processes to co-operate.

i) Information Sharing

ii) Computation Speedup

iii) Modularity

iv) Conve.

6 Inter Process Communication

It is a mechanism

IPC Modes

(i) Shared Memory :- In this mode, multiple process share a common memory space. They can read / write data directly on that memory.
* fast communication & also need synchronization.

(ii) Message Passing :- In this mode, processes communicate by sending & receiving messages through the OS.

Easy & Safe [no shared memory issue but slower than Shared memory.

Unit 3

Critical Section ~~?~~

Critical Section: It is a code segment where process access shared resources (likefi-les, variable, pointers) that need synchroniz-ed exclusive access to prevent data corr

→ It is part of the program where shared resources are accessed by various process.

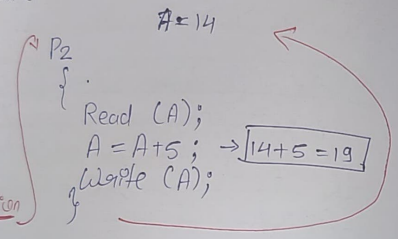
Placed where variables, ~~resources~~

~~*~~

If both CS run together it create ~~may~~ prob. It is known as race condition

Ex of Race Condition

A = 7 Shared Variable



P1 & P2 → Concurrent Process → Same Time → Shared Variable → Access

↓

Data Inconsistency

A	P1	P2
7	7*2 = 14	14+5 = 19 → Synchronized Way running
7	14	12 → ASynchronized.

Process Synchronization Unit 3

Why we Require Process Synchronization

Two Type of Process Categeories :

i) Independent Process : When one Process is execution It does not affect other Execution Processes.

ii) Cooperative Process : It Affects other Executing Processes.

* RACE CONDITION : It occurs when two or more Processes are executed at the same time, Not Scheduled In Proper Sequence & Not executed in Critical Section Correctly. Which Causes "DATA In CONSISTENCY & DATA LOSS"

* Process Synchronization : It helps To maintain Shared data Consistency & Cooperating Process

• Execution when process execute it should be ensured That concurrent access to Shared data not create inconsistencies.

Unit 4 Bounded Buffer Prob.

PRODUCER CONSUMER PROBLEM

The Producer-consumer problem is Synchroniz-ation Problem in an OS where one process called the producer creates data & puts it into a buffer & another process called the consumer takes the data from the buffer & uses it. The system must control both processes so that the buffer does not become full or empty at the wrong time.

* Producer: Produces data or items & stores them in buffer.

* Consumer: Takes items from the buffer & uses them

* The Operating System uses synchronization method (like semaphore or mutex) to make sure both processes work properly.